applied sciences

MDPI

Article

Curvature-Constrained Motion Planning Method for Differential-Drive Mobile Robot Platforms

Rudolf Krecht [1,*] and Áron Ballagi [2]

[1] Department of Defence Innovation and Critical Infrastructures, Széchenyi István University, H-9026 Győr, Hungary
[2] Department of Automation and Robotics, Széchenyi István University, H-9026 Győr, Hungary; ballagi@ga.sze.hu
* Correspondence: krecht.rudolf@ga.sze.hu

Abstract

Compact heavy-duty skid-steer robots are increasingly used for city logistics and intralogistics tasks where high payload capacity and stability are required. However, their limited maneuverability and non-negligible turning radius challenge conventional waypoint-tracking controllers that assume unconstrained motion. This paper proposes a curvature-constrained trajectory planning and control framework that guarantees geometrically feasible motion for such platforms. The controller integrates an explicit curvature limit into a finite-state machine, ensuring smooth heading transitions without in-place rotation. The overall architecture integrates GNSS-RTK and IMU localization, modular ROS 2 nodes for trajectory execution, and a supervisory interface developed in Foxglove Studio for intuitive mission planning. Field trials on a custom four-wheel-drive skid-steer platform demonstrate centimeter-scale waypoint accuracy on straight and curved trajectories, with stable curvature compliance across all tested scenarios. The proposed method achieves the smoothness required by most applications while maintaining the computational simplicity of geometric followers. Computational simplicity is reflected in the absence of online optimization or trajectory reparameterization; the controller executes a constant-time geometric update per cycle, independent of waypoint count. The results confirm that curvature-aware control enables reliable navigation of compact heavy-duty robots in semi-structured outdoor environments and provides a practical foundation for future extensions.

Keywords: skid-steer robot; curvature constraint; trajectory tracking; finite-state machine; GNSS-RTK localization

Academic Editor: Alessandro Gasparetto

Received: 17 November 2025
Revised: 23 December 2025
Accepted: 25 December 2025
Published: 28 December 2025

1. Introduction

Autonomous transportation technologies are redefining the interaction between goods, materials, and infrastructure in industrial and urban environments [1,2]. Although much of the early focus in autonomy has centered on large-scale road vehicles, recent developments have highlighted the growing importance of compact, task-specific mobile systems operating in semi-structured domains. In parallel, the rise of micromobility and small-scale autonomous platforms has been recognized as a key enabler of sustainable transportation, reducing energy consumption, emissions, and spatial footprint while improving the flexibility of urban logistics and last-mile delivery [3]. Beyond single-robot navigation, city-logistics deployment increasingly requires fleet-level coordination and routing integrated with motion planning under real-world uncertainty [4]. Among these, heavy-duty skid-steer robots have emerged as robust solutions for logistics tasks that demand

compact dimensions with high load capacity, and resilience against uneven terrain or environmental variability.

Intralogistics and infrastructure maintenance represent two domains in which such platforms provide immediate benefits. Intralogistics involves the autonomous handling and transportation of materials within factories, ports, and construction sites, where precise motion and repeated task execution are required in semi-constrained environments. Similarly, infrastructure-oriented robots are being increasingly deployed for inspection, surveying, and material handling in outdoor areas that lack structured navigation features. When designed for short-distance, energy-efficient operations, such compact robots contribute to sustainable mobility ecosystems by complementing larger autonomous vehicles and reducing the environmental impact of frequent low-payload transport tasks. In both domains, compact heavy-duty robots offer the advantage of maneuverability within confined areas while providing sufficient strength to transport or manipulate substantial payloads. However, achieving such performance introduces challenges related to kinematic feasibility and control smoothness [5].

Heavy-duty skid-steer platforms are typically designed with an emphasis on stability, traction, and mechanical robustness rather than agility. Their limited steering capability and non-negligible turning radius make it impossible to perform in-place rotations or instantaneous heading changes. As a consequence, feasible trajectories are curvature-bounded, and controllers that neglect these constraints may issue infeasible commands or induce wheel slip. Classical differential-drive controllers, which assume free rotational capability, often fail to ensure curvature continuity and can produce discontinuous or unstable motion profiles on such platforms. A reliable motion control framework for these robots must therefore explicitly consider the geometric and physical limits imposed by the platform design.

The increasing deployment of skid-steer mobile platforms across industrial, agricultural, and inspection domains has intensified research on motion planning and control strategies tailored to the unique properties of this locomotion paradigm. Skid-steer mobile robots (SSMRs) exhibit fundamentally different turning characteristics than differential-drive or Ackermann-steered systems, as steering is achieved exclusively through differential wheel speeds. This mode of locomotion inherently induces wheel slip, lateral skidding, and strong terrain-dependent interactions. As vehicle mass and payload capacity increase, the resulting shear forces impose strict practical limits on the feasible turning radius; exceeding these limits can lead to structural stress, tire degradation, or unstable motion. Consequently, explicit curvature constraints become essential in the motion control of heavy-duty SSMRs.

A variety of modelling and control approaches have been proposed to address these issues. A learning-based modelling framework was introduced in [6], capturing both slip dynamics and energy consumption for improved motion planning accuracy. Classical modelling work such as [7] establishes detailed kinematic and dynamic formulations for four-wheel skid-steer platforms, highlighting stability limitations and the non-holonomic structure induced by differential speed control. More recent developments include a curved-path-following controller based on backstepping that accounts for both kinematic and steering dynamics [8]. These contributions demonstrate that SSMRs cannot assume zero turning radius or arbitrarily large yaw rates, reinforcing the need for controllers that explicitly respect curvature or turning-radius bounds.

Trajectory planning methods for mobile robots often overlook the physical turning-radius constraint, despite its critical importance for vehicles that cannot rotate in place without inducing substantial lateral slip. Classical Dubins curves provide a minimum-turning-radius model but remain restricted to simple geometric primitives and do not

address smooth, continuous trajectories or closed-loop tracking [9]. More recent research explicitly integrates curvature feasibility. A curvature-constrained vector-field formulation for non-holonomic robots was proposed in [10], ensuring bounded-curvature trajectories and guaranteeing convergence under actuation limits. Similarly, a Hamilton–Jacobi PDE-based formulation for optimal motion generation of curvature-bounded vehicles has been developed in [11]. A recent survey of sampling-based motion planners [12] further reveals that many widely used planners inadequately address curvature and non-holonomic constraints, thereby motivating the integration of curvature bounds into both global path planning and local control layers.

In addition to geometric feasibility, real-world trajectory execution requires speed profiles consistent with curvature and dynamic constraints. This is especially relevant for heavy-duty platforms, where abrupt changes in speed or curvature may lead to excessive drivetrain loading, wheel slip, or payload oscillation. The S-curve trajectory planning method presented in [13] incorporates curvature radius and centripetal acceleration constraints, illustrating the strong coupling between curvature and velocity. The FDSPC framework [14] further extends this line of work by generating G^2-continuous trajectories with integrated speed planning for bounded-curvature vehicles. For continuous-curvature path design, clothoid-based smoothing is a well-established approach for differential-drive robots under kinematic constraints, providing curvature continuity while remaining computationally tractable [15]. These contributions underline the need for motion controllers that jointly regulate heading, curvature, and velocity while maintaining simple deployment for field deployment.

Although the literature offers extensive insights into skid-steer modelling, curvature-constrained planning, and smooth trajectory generation, several gaps remain for compact heavy-duty SSMRs operating in semi-structured outdoor logistics environments. Much of the existing research relies on idealized vehicle models such as Dubins cars or planar unicycles, which do not accurately represent the traction conditions, slip behavior, or turning limitations of high-mass skid-steer platforms. Furthermore, several curvature-aware planning approaches impose computational complexity.

At the same time, advances in localization and software infrastructure have made the deployment of these robots increasingly accessible. Global Navigation Satellite Systems with Real-Time Kinematic (GNSS-RTK) enable centimeter-level localization in open environments, while inertial measurement units (IMUs) provide complementary short-term motion estimates. The Robot Operating System 2 (ROS 2) offers a deterministic, modular middleware for integrating sensing, planning, and control components. Browser-based tools such as Foxglove Studio [16] extend this ecosystem by providing intuitive mission supervision and trajectory editing capabilities, facilitating operator oversight even in remote or distributed setups.

This paper presents a generalized curvature-constrained trajectory execution framework tailored for compact heavy-duty skid-steer robots. The proposed approach introduces an explicit curvature-bounded control law integrated into a discrete event-driven state machine that governs waypoint alignment, progression, and mission-driven stopping behaviors. The architecture combines curvature-aware control with modular localization, trajectory management, and supervision components within the ROS 2 framework. In addition, it integrates a browser-based operator interface developed in Foxglove Studio, allowing users to define waypoints, assign speeds, and monitor mission execution in real time. The system has been validated in field conditions, demonstrating centimeter-level accuracy and stable curvature compliance in diverse operating scenarios.

The present work addresses these gaps by introducing a curvature-constrained trajectory execution framework tailored specifically to compact heavy-duty skid-steer robots. The proposed system embeds curvature bounds directly into the control law, em-

ploys a finite-state machine for robust mission execution, and integrates seamlessly with GNSS–RTK–IMU localization and ROS 2-based supervision. The method prioritizes geometric feasibility, computational simplicity, and practical deployability, and its effectiveness is demonstrated through field experiments.

This article extends our earlier conference paper [17]. While [17] focused on a specific application scenario and platform-level implementation, the present work provides a generalized and hardware-independent formulation of curvature-constrained trajectory execution. In addition, the current manuscript introduces (i) an explicit curvature-bounded control formulation with geometric interpretation of the minimum turning radius, (ii) a systematic comparison against baseline controllers without curvature constraints, and (iii) an expanded field evaluation using rosbag-based analysis across multiple trajectory patterns and waypoint densities. These additions significantly extend the scope, analysis depth, and reproducibility beyond the conference version.

2. Materials and Methods

This section presents the architecture, sensing, and control framework developed for curvature-constrained trajectory execution on compact heavy-duty skid-steer robots. The system integrates real-time localization, modular communication under ROS 2, a curvature-aware motion controller, and a web-based human–machine interface for mission supervision. Unlike clothoid-based smoothing methods that explicitly construct G^2 paths, the present work focuses on online curvature feasibility during execution via curvature saturation, combined with optional waypoint resampling for spacing control. The overall design prioritizes geometric feasibility, real-time responsiveness, and operator interpretability over algorithmic complexity.

2.1. System Architecture

The robot platform was designed to balance structural robustness with modularity and serviceability. Figure 1 illustrates the main hardware and software components. Each of the four brushless drive motors is controlled by an electronic speed controller (ESC) interfaced via Controller Area Network (CAN) bus. Two independent CAN segments are managed by a central industrial PC to ensure fault isolation and deterministic communication. The compute unit hosts ROS 2 nodes for localization, control and supervision, while a secondary embedded GPU computer may be used for perception or data entry tasks. Both systems communicate over a high-bandwidth local network using a router that also provides remote access through a secure virtual private network.

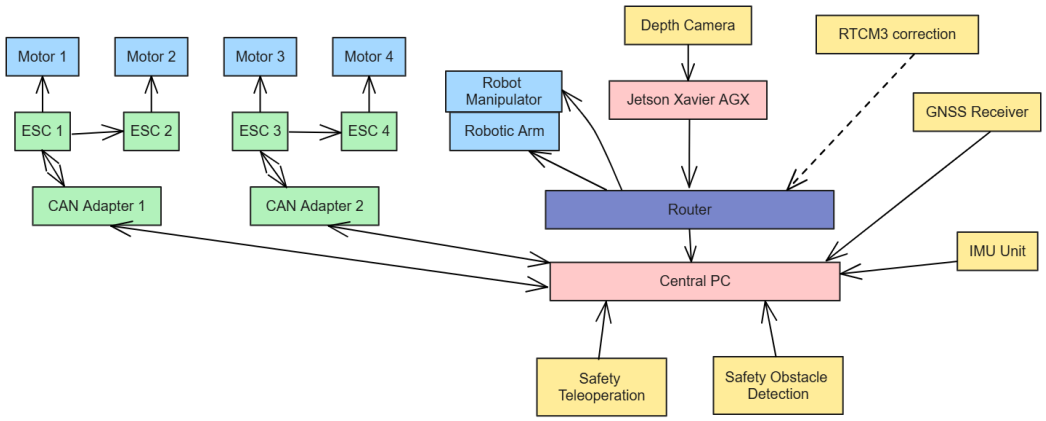

Figure 1. System architecture of the compact heavy-duty skid-steer robot, showing the integration of drive, sensing, compute, and supervision subsystems. Arrows indicate the primary data flow between subsystems.

Inertial data are provided by a six-axis IMU directly connected to the industrial PC through a serial interface. The global position is obtained from a GNSS-RTK (Drotek DP0601) receiver capable of centimeter-level accuracy under open-sky conditions. Real-time corrections are received through the Networked Transport of RTCM via Internet Protocol (NTRIP). The localization used by the controller does not rely on a tightly coupled GNSS–IMU fusion filter. Instead, global position is obtained directly from the GNSS receiver, while orientation (yaw) is provided by the onboard IMU. These signals are combined via a ROS 2 transform (`/tf`) to provide the robot pose used by the controller. No additional state estimation or filtering beyond sensor-level processing is applied. This design choice prioritizes transparency and allows the controller behavior to be evaluated independently of a specific fusion algorithm.

The control loop runs at 10 Hz under ROS 2 using a single-threaded executor, ensuring deterministic state-machine transitions and bounded latency.

2.2. Human–Machine Interface

A custom supervision interface was developed using Foxglove Studio, a web-based visualization and mission control tool compatible with ROS 2. The interface allows the operator to author, edit, and monitor trajectories directly on a map view, as shown in Figure 2. The panel was implemented as a Foxglove extension written in React and TypeScript and communicates with the robot through a WebSocket bridge. It provides access to the robot's real-time state, published topics, and available ROS services.

Waypoints are represented as a `geometry_msgs/msg/PoseArray` message, while associated speed values are transmitted in a corresponding `std_msgs/msg/Float32MultiArray`. Operator commands, including start, stop, or reset actions, are implemented via standard `std_srvs/srv/Trigger` services. The graphical interface allows trajectory definition through direct map interaction: each click adds a waypoint, which can be repositioned or removed dynamically. Speed values can be adjusted per waypoint to encode velocity profiles or task semantics such as temporary stops. Once the configuration is finalized, the panel publishes the updated waypoint array and speed sequence to the controller node.

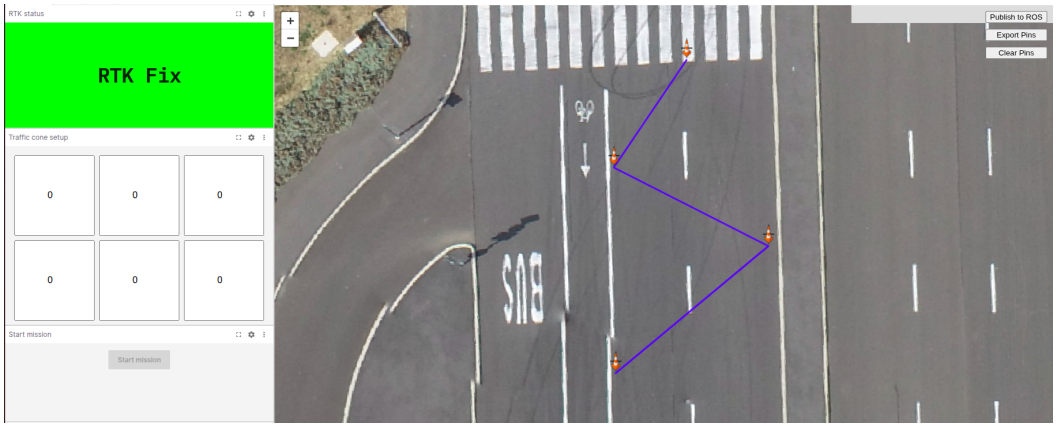

Figure 2. Custom Foxglove Studio interface for waypoint definition and mission supervision. The operator can edit paths and issue control commands through a browser-based interface.

This design choice ensures platform-independent deployment: the panel does not require a local ROS installation, which simplifies multi-user access and supports remote operation. The use of Foxglove's native React environment enables responsive visualization and low-latency updates synchronized with ROS 2 communication.

2.3. Trajectory Definition, Resampling, and Execution

The system follows a waypoint-based trajectory definition strategy. A trajectory is represented as an ordered set of waypoints (x_i, y_i, θ_i) in global coordinates, accompanied by a corresponding speed profile $\{v_i\}$. Each waypoint defines a desired position and orientation, while its assigned speed value determines the target linear velocity in the vicinity of that waypoint. Speed values may vary continuously or include explicit zeros, indicating mission-driven stops for task execution or safety checks.

In addition to the operator-defined waypoint list, the framework optionally performs waypoint resampling (densification) to obtain a desired spatial spacing s between consecutive points (e.g., $s \in \{0.5, 1.0, 2.0\}$ m in the evaluation). For each consecutive pair of waypoints $W_i = (x_i, y_i)$ and $W_{i+1} = (x_{i+1}, y_{i+1})$, we compute the segment length $\ell_i = \|W_{i+1} - W_i\|$ and insert

$$n_i = \max\left(\left\lceil \frac{\ell_i}{s} \right\rceil - 1,\, 0\right) \tag{1}$$

intermediate points by linear interpolation

$$W_{i,k} = W_i + \frac{k}{n_i + 1}(W_{i+1} - W_i), \quad k = 1, \ldots, n_i. \tag{2}$$

The corresponding speeds are interpolated linearly between v_i and v_{i+1}. This resampling step improves geometric fidelity for sparse waypoint sets, while kinematic feasibility is guaranteed online by the curvature saturation in the controller.

The resulting waypoint sequence is published as discrete targets for the controller, which sequentially processes them in real time.

2.4. Kinematic Model and Curvature Constraints

The robot's motion is modeled by a standard nonholonomic unicycle formulation [18,19]:

$$\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega, \quad \omega = v\kappa, \tag{3}$$

where (x, y, θ) denotes the robot's pose, v the linear velocity, and ω the angular velocity. The instantaneous curvature κ is defined as the ratio of angular to linear velocity and is bounded by the platform's physical constraints:

$$|\kappa| \leq \kappa_{\max} = \frac{1}{R_{\min}}. \tag{4}$$

The controller explicitly enforces this bound at every update cycle, ensuring that the robot's commanded motion never exceeds its geometric turning limits. This approach allows continuous forward-facing trajectories even in scenarios where abrupt direction changes would otherwise require in-place rotation.

The kinematic model neglects dynamic effects such as mass, wheel–terrain interaction, lateral slip, and load transfer. These effects do not appear explicitly as disturbance terms in the model and no formal disturbance rejection or robustness proof is claimed. In practice, their influence manifests only as deviations in the measured pose, which are corrected indirectly by the closed-loop geometric feedback and by conservative curvature limiting. The present formulation therefore focuses on geometric feasibility and bounded curvature rather than explicit dynamic compensation.

2.5. Curvature-Constrained Control Framework

The control strategy is implemented as an event-driven finite-state machine composed of three operational states [20]: *Align*, *Goto*, and *Pause*. In the *Align* state, the robot gradually adjusts its heading toward the next waypoint while moving at a reduced speed, using a curvature-bounded forward arc rather than an in-place rotation. Once the heading error falls below a predefined tolerance, the system transitions to the *Goto* state, during which the robot advances toward the target while continuously regulating its curvature within allowable limits. When a waypoint with zero assigned speed is reached, the controller switches to the *Pause* state, halts the robot, executes any pending service commands, and resumes motion after a parameterized delay.

To ensure smooth and kinematically feasible motion, angular velocity is computed using a curvature-based control strategy. Let (x, y, θ) denote the current pose of the robot, and let (x_w, y_w) be the position of the target waypoint. The Euclidean distance to the waypoint is given by:

$$d = \sqrt{(x_w - x)^2 + (y_w - y)^2} \tag{5}$$

The desired heading is computed using the two-argument arctangent function [18]:

$$\theta_d = \text{atan2}(y_w - y, x_w - x) \tag{6}$$

To compute a smooth angular error, we use the wrapped angle difference:

$$\Delta\theta = \text{atan2}(\sin(\theta_d - \theta), \cos(\theta_d - \theta)) \tag{7}$$

From the heading error and distance, we compute an instantaneous curvature κ:

$$\kappa = \frac{\Delta\theta}{\max(d, \varepsilon)} \tag{8}$$

where ε is a small positive constant used to prevent division by zero at short distances.

To respect the robot's physical turning limits, the curvature is clamped to a maximum value $\kappa_{\max} = \frac{1}{R_{\min}}$, where $R_{\min}$ is the minimum turning radius [19]:

$$\kappa \leftarrow \text{clip}(\kappa, -\kappa_{\max}, \kappa_{\max}) \tag{9}$$

where $\text{clip}(\cdot)$ denotes a saturation operator:

$$\text{clip}(x, x_{\min}, x_{\max}) = \min(\max(x, x_{\min}), x_{\max}). \tag{10}$$

It is important to mention that $\kappa_{\max}$ is not claimed to prevent slip, it reduces the impact by avoiding aggressive, tight turns. The minimum turning radius $R_{\min}$ was defined geometrically as a design parameter, independent of any dynamic or physical stress model. The objective was to ensure that the robot can reach laterally offset waypoints using smooth forward motion without requiring in-place rotation. After that, the value was tuned by slight changes based on empirical observation on one surface type, asphalt.

Specifically, $R_{\min}$ was selected as the smallest radius of a circular arc that connects the robot's initial pose to a target waypoint located at a longitudinal offset Δx and a lateral offset Δy. Assuming a forward circular arc, the required turning radius can be computed as

$$R_{\min} = \frac{\Delta x^2 + \Delta y^2}{2\,\Delta y}, \tag{11}$$

which corresponds to the radius of the circle passing through the start point and the target waypoint with an initial tangent aligned to the forward direction.

The angular velocity command is obtained as

$$\omega = v\kappa. \tag{12}$$

This formulation yields smooth angular behavior proportional to the alignment error while respecting the physical curvature constraint. The state machine transitions are triggered by distance and heading thresholds, ensuring deterministic progress along the waypoint sequence.

The algorithm based on the aforementioned control rules is summarized in Algorithm 1. It outlines the finite-state decision logic and the curvature-constrained computation of the angular velocity command. The parameters used during the field tests are concluded by Table 1.

Algorithm 1: Finite-state curvature-constrained trajectory follower.

Data: Current pose (x, y, θ), next waypoint (x_w, y_w), assigned speed v_w
Result: Velocity command (v, ω)
Compute $dx \leftarrow x_w - x$, $dy \leftarrow y_w - y$;
Compute distance $d \leftarrow \sqrt{dx^2 + dy^2}$;
Compute desired heading $\theta_d \leftarrow \text{atan2}(dy, dx)$;
Compute wrapped heading error $\Delta\theta \leftarrow \text{atan2}(\sin(\theta_d - \theta), \cos(\theta_d - \theta))$;
if *state* = *ALIGN* **then**
  if $|\Delta\theta| < \varepsilon_\theta$ **then**
    state $\leftarrow$ GOTO
  end
  else
    $v \leftarrow v_{\text{align}}$;
    $\kappa \leftarrow \text{clip}\left(\frac{\Delta\theta}{\max(d,\varepsilon)}, -\kappa_{\max}, \kappa_{\max}\right)$;
    $\omega \leftarrow v\kappa$;
  end
else if *state* = *GOTO* **then**
  $v \leftarrow v_w$;
  $\kappa \leftarrow \text{clip}\left(\frac{\Delta\theta}{\max(d,\varepsilon)}, -\kappa_{\max}, \kappa_{\max}\right)$;
  $\omega \leftarrow v\kappa$;
  if $d < \varepsilon_d$ **then**
    if $v_w = 0$ **then**
      state $\leftarrow$ PAUSE
    end
    else
      advance to next waypoint;
    end
  end
else if *state* = *PAUSE* **then**
  stop motors; execute task service;
  wait t_{pause};
  state $\leftarrow$ ALIGN;

Table 1. Main controller parameters used in the field experiments.

Parameter	Value
Control update rate	10 Hz
Nominal speed v_{nom}	0.6 m/s
Alignment speed v_{align}	0.25 m/s
Minimum speed near goal $v_{\min}$	0.15 m/s
Waypoint reach radius ε_d	0.15 m
Heading tolerance ε_θ	0.4 rad
Distance regularizer ε	0.25 m
Minimum turning radius $R_{\min}$	0.10 m
Maximum curvature $\kappa_{\max}$	10.0 rad/m
Tangent blending radius	0.8 m

2.6. Stability Considerations

Assuming bounded curvature and positive forward velocity, the closed-loop heading error dynamics can be approximated by

$$\dot{\Delta\theta} \approx -kv\Delta\theta, \quad k = \frac{1}{\max(d, \varepsilon)}, \tag{13}$$

which guarantees an exponential convergence of $\Delta\theta$ towards zero. As a consequence, both heading and position errors remain bounded and asymptotically converge with nominal localization accuracy. The curvature constraint further ensures that motion remains geometrically feasible and mechanically smooth throughout the trajectory. The approximation in Equation (13) represents a local analysis of the unsaturated regime of the controller, where the curvature command is given by $\kappa = \Delta\theta / \max(d, \varepsilon)$. Under forward motion ($v > 0$), $d > \varepsilon$, and sufficiently small heading errors, this yields locally exponential convergence of the heading error. When curvature saturation is active, the same linearized dynamics no longer applies; however, saturation guarantees bounded curvature and angular velocity, ensuring safe and geometrically feasible motion. A global stability proof under saturation, slip, and state-machine switching is beyond the scope of this work.

2.7. Baseline Controllers for Comparison

To quantify the effect of explicit curvature saturation, we compare the proposed curvature-clipped point-to-point (P2P) controller against two geometric baselines:

2.7.1. Baseline A: Unclipped Point-to-Point (P2P)

This baseline uses the same heading-to-waypoint control law as the proposed method, but removes the curvature saturation step. In particular,

$$\kappa = \frac{\Delta\theta}{\max(d, \varepsilon)}, \qquad \omega = v\kappa, \tag{14}$$

which may command curvature values exceeding $\kappa_{\max}$.

2.7.2. Baseline B: Pure Pursuit

We implement a standard Pure Pursuit follower on the same waypoint polyline. Given a lookahead distance L, the algorithm selects a lookahead point on the reference path and computes curvature as

$$\kappa = \frac{2\sin(\alpha)}{L}, \qquad \omega = v\kappa, \tag{15}$$

where α is the heading angle between the robot's forward axis and the lookahead point. Unless stated otherwise, Pure Pursuit does not explicitly enforce $\kappa_{\max}$ (as is typical in standard implementations).

3. Results

The proposed curvature-constrained trajectory control framework was validated through field experiments. The objective of these evaluations was to verify that the controller could accurately follow curvature-feasible waypoint sequences, maintain stability during state transitions, and achieve smooth motion under the physical turning constraints of a compact heavy-duty skid-steer robot.

In addition to the proposed curvature-constrained follower (curvature-clipped point-to-point, clipped P2P), two baseline controllers were evaluated on the same platform: (i) an unclipped P2P controller (identical logic but without curvature saturation), and (ii) a Pure Pursuit controller (PPC). All runs were recorded to `rosbag2` for offline analysis and direct reproducibility.

Three representative path patterns were tested: straight, curved, and zig-zag. To study waypoint-density sensitivity, each pattern was executed with three waypoint spacings: sparse (2.0 m), medium (1.0 m), and dense (0.5 m). Figure 3 summarizes the resulting field trajectories across controllers, patterns, and densities.

For each controller–trajectory–density combination, one representative field run was analyzed; the focus of the evaluation is comparative controller behavior under identical conditions rather than statistical averaging. Runs were carried out with the same environmental conditions.

Trajectory tracking accuracy was evaluated using the recorded robot pose (from `/tf`, frame `tyr_base_link`) and the waypoint set (from `/waypoint_markers`). For each waypoint $W_i = (x_i, y_i)$, we compute the closest-approach distance to the executed trajectory samples $p_k = (x_k, y_k)$:

$$e_i = \min_k \|p_k - W_i\|_2, \tag{16}$$

and report waypoint proximity MAE, RMSE, and maximum error over the waypoint set. In addition, we report curvature statistics computed from the recorded trajectory to quantify curvature demand and to highlight violations in the unclipped baseline.

The comparison demonstrates that the curvature-clipped controller provides the most consistent performance on curvature-demanding trajectories while maintaining bounded curvature. In the Curved–Medium configuration, the proposed method achieves MAE $= 0.0096$ m, compared to 0.165 m for the unclipped baseline and 0.056 m for Pure Pursuit (Table 2). Curvature-related differences are further illustrated by the temporal evolution and curvature comparison plots for a representative Curved–Medium run (Figures 4 and 5), where the unclipped baseline exhibits large curvature peaks while the proposed controller remains bounded.

Overall, the field experiments confirm that explicitly enforcing a curvature constraint improves reliability and repeatability on curved and zig-zag trajectories, while maintaining comparable performance on straight segments. The waypoint-density study further indicates that waypoint spacing influences tracking behavior; overly sparse waypoint sets can induce larger heading transients, while very dense waypoint sets increase the frequency of waypoint transitions in the finite-state logic.

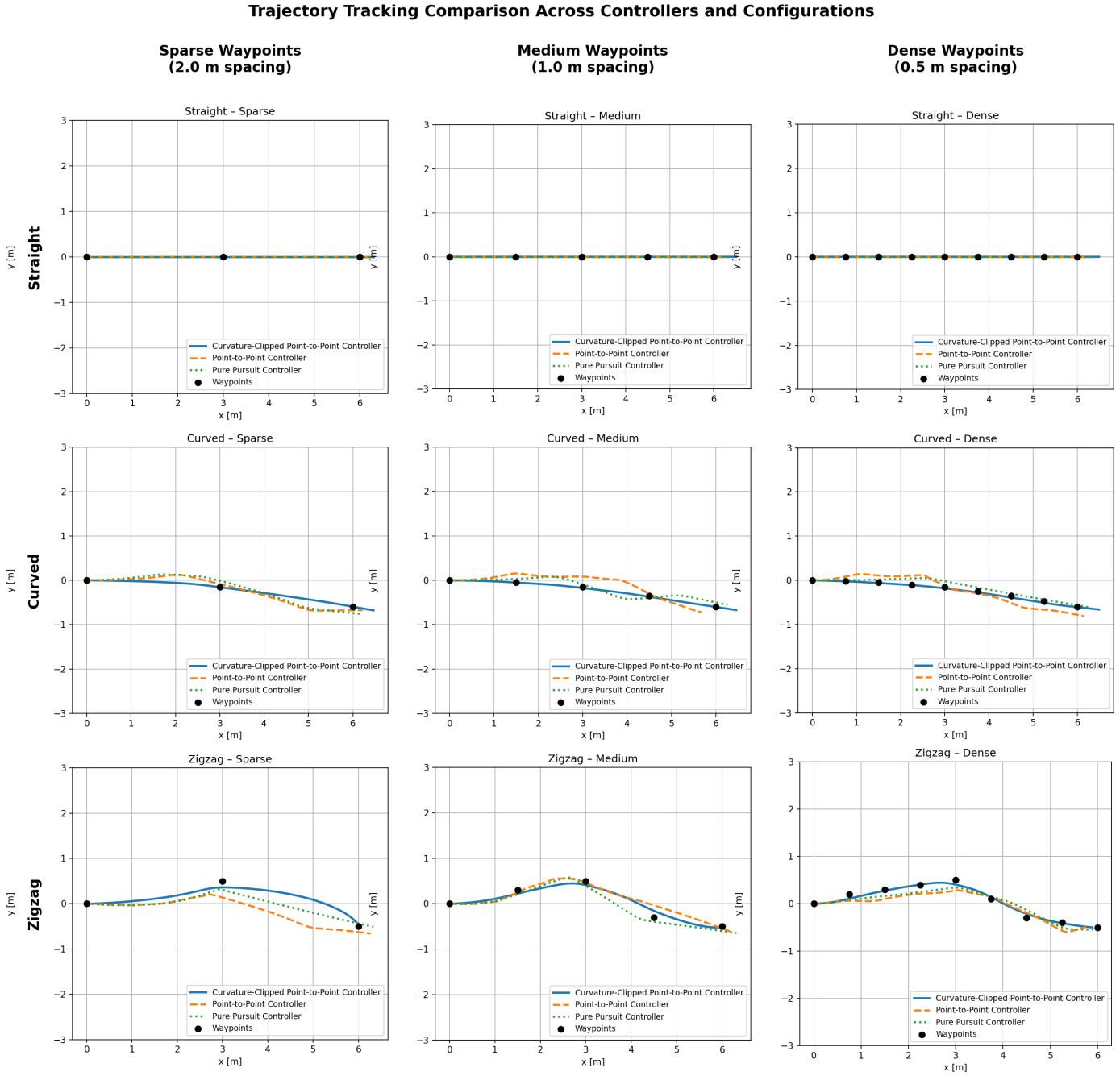

Figure 3. Field trajectory tracking comparison across controllers and waypoint densities. Rows: straight, curved, and zig-zag patterns. Columns: waypoint spacing (sparse: 2.0 m, medium: 1.0 m, dense: 0.5 m). Black markers indicate waypoints.

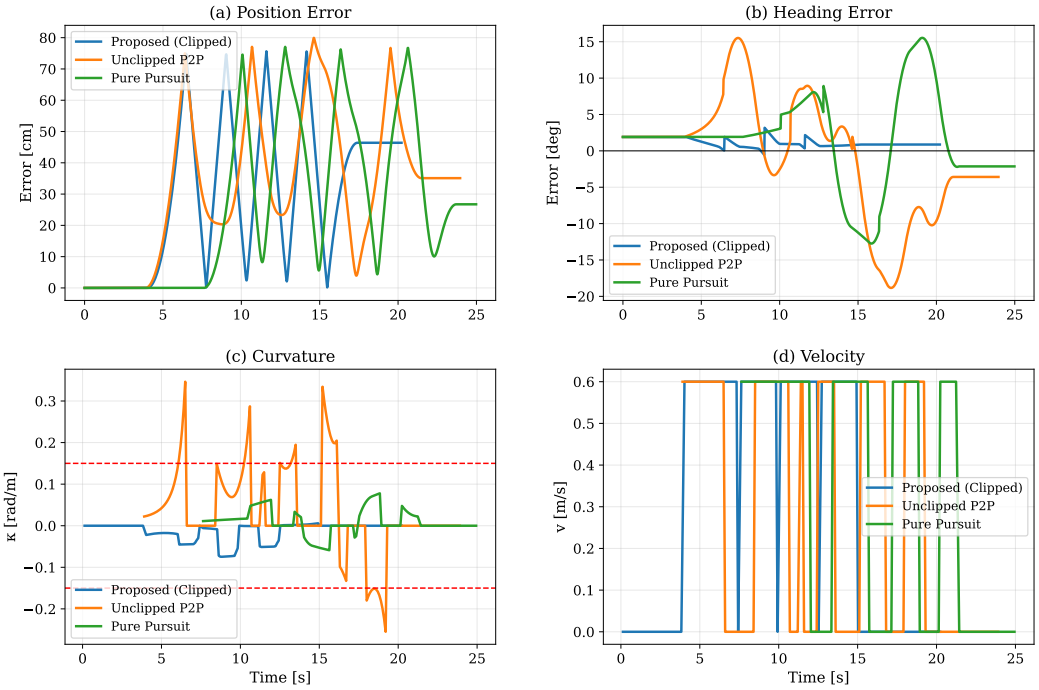

Figure 4. Representative field run (Curved–Medium): temporal evolution of (**a**) waypoint tracking error, (**b**) heading deviation, (**c**) curvature, and (**d**) velocity profile for the three controllers.

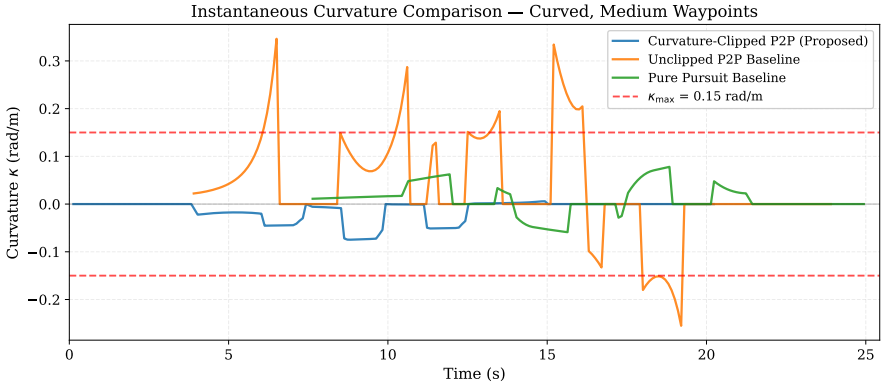

Figure 5. Representative field run (Curved–Medium): instantaneous curvature comparison across controllers. The plot highlights curvature peaks in the unclipped baseline and bounded behavior in the proposed curvature-clipped follower.

Table 2. Field-test comparison of waypoint proximity errors and peak curvature computed from recorded rosbags. Waypoint error is the closest-approach distance to each waypoint.

Pattern	Density	Controller	MAE [m]	RMSE [m]	Max [m]	κ_{max} [rad/m]
Straight	Sparse (2.0 m)	Proposed (clipped P2P)	0.002100	0.003468	0.006000	0.000000
Straight	Sparse (2.0 m)	Unclipped P2P	0.002100	0.003468	0.006000	0.000000
Straight	Sparse (2.0 m)	Pure Pursuit	0.002100	0.003468	0.006000	0.000000
Straight	Medium (1.0 m)	Proposed (clipped P2P)	0.002260	0.003003	0.005000	0.000000
Straight	Medium (1.0 m)	Unclipped P2P	0.003660	0.004650	0.006000	0.000000
Straight	Medium (1.0 m)	Pure Pursuit	0.003660	0.004650	0.006000	0.000000
Straight	Dense (0.5 m)	Proposed (clipped P2P)	0.003178	0.003728	0.006000	0.000000
Straight	Dense (0.5 m)	Unclipped P2P	0.002033	0.003466	0.006000	0.000000
Straight	Dense (0.5 m)	Pure Pursuit	0.002033	0.003466	0.006000	0.000000

Table 2. *Cont.*

Pattern	Density	Controller	MAE [m]	RMSE [m]	Max [m]	κ_{max} [rad/m]
Curved	Sparse (2.0 m)	Proposed (clipped P2P)	0.004472	0.005501	0.007343	0.102500
Curved	Sparse (2.0 m)	Unclipped P2P	0.047408	0.058222	0.076380	0.168062
Curved	Sparse (2.0 m)	Pure Pursuit	0.091530	0.112111	0.139169	0.102136
Curved	Medium (1.0 m)	Proposed (clipped P2P)	0.009629	0.014221	0.023963	0.077444
Curved	Medium (1.0 m)	Unclipped P2P	0.165289	0.209914	0.350934	0.364858
Curved	Medium (1.0 m)	Pure Pursuit	0.056396	0.066151	0.100661	0.084151
Curved	Dense (0.5 m)	Proposed (clipped P2P)	0.017572	0.022114	0.039036	0.101819
Curved	Dense (0.5 m)	Unclipped P2P	0.116687	0.136883	0.206604	0.417904
Curved	Dense (0.5 m)	Pure Pursuit	0.063639	0.078085	0.144804	0.099748
Zig-zag	Sparse (2.0 m)	Proposed (clipped P2P)	0.082165	0.101541	0.139862	0.146017
Zig-zag	Sparse (2.0 m)	Unclipped P2P	0.158604	0.215108	0.351079	0.205114
Zig-zag	Sparse (2.0 m)	Pure Pursuit	0.085341	0.117110	0.192703	0.013977
Zig-zag	Medium (1.0 m)	Proposed (clipped P2P)	0.079030	0.090481	0.132780	0.000000
Zig-zag	Medium (1.0 m)	Unclipped P2P	0.075050	0.117253	0.252388	0.286894
Zig-zag	Medium (1.0 m)	Pure Pursuit	0.074337	0.083230	0.098534	0.094101
Zig-zag	Dense (0.5 m)	Proposed (clipped P2P)	0.049435	0.063927	0.106754	0.000000
Zig-zag	Dense (0.5 m)	Unclipped P2P	0.142474	0.161142	0.241579	0.300231
Zig-zag	Dense (0.5 m)	Pure Pursuit	0.107549	0.118562	0.159331	0.112070

Limitations

The experiments also highlighted several practical limitations that inform future development. The robot requires stable GNSS reception and access to RTK correction data; performance degrades in areas affected by multipath interference or limited satellite visibility. The controller further assumes that all waypoints are geometrically reachable under the robot's curvature constraints. If waypoints are placed too close together, behind the vehicle, or require curvature exceeding the minimum turning radius, the resulting trajectory may become infeasible, currently necessitating manual adjustment of the waypoint layout.

From an evaluation perspective, the primary quantitative metric used in this work is the waypoint proximity error, defined as the closest-approach distance between the executed trajectory and each waypoint. While this metric directly reflects the controller's ability to satisfy operator-defined navigation goals, it does not explicitly capture continuous path deviation between waypoints, particularly for sparse waypoint configurations. To address this limitation, the waypoint-based evaluation is complemented by time-resolved cross-track and heading error plots, as well as curvature profiles for representative runs, which reveal transient behavior and curvature feasibility during execution.

Finally, although the curvature constraint enforces geometrically smooth motion, the current control formulation does not explicitly model dynamic effects such as lateral slip, load transfer, or terrain-dependent traction variations. These effects are treated implicitly through closed-loop pose feedback and conservative curvature limits. Incorporating slip-aware or dynamic extensions of the model, as well as automated waypoint feasibility checks, constitutes an important direction for future work.

4. Discussion and Conclusions

The results demonstrate that curvature-constrained trajectory execution provides a practical and reliable solution to guide compact heavy-duty skid-steer robots under geometric turning limits. By embedding curvature feasibility directly into the control law and employing a discrete event-driven state machine, the system achieves continuous and stable motion without requiring complex optimization or heavy computation. The experimental results confirm that the robot can follow manually defined trajectories with centimeter-scale accuracy while maintaining smooth and physically consistent motion, validating the suitability of the proposed framework for real-world deployment.

A key advantage of the presented approach is its balance between geometric rigor and implementation simplicity. The explicit curvature bound ensures that all commanded motions remain feasible for the vehicle's kinematic configuration, thereby preventing infeasible rotational maneuvers or wheel slip that could otherwise arise on high-friction surfaces. The finite-state structure provides intuitive behavioral segmentation—alignment, forward motion, and pause, allowing for clear supervision and integration with task-level logic. The framework is fully compatible with standard ROS 2 infrastructures, which facilitates its reuse across different robotic platforms and control stacks.

The developed operator interface contributes to the system's practical usability by providing an intuitive, map-based method for defining and editing trajectories. Its web-based deployment and integration with ROS 2 communication channels enable remote supervision and collaborative mission design, aligning with current trends toward human-in-the-loop autonomous systems. This integration is particularly relevant for outdoor robotics and industrial logistics scenarios, where situational awareness and rapid mission adjustment are critical.

The achieved positional accuracy, limited primarily by GNSS-RTK precision, indicates that the proposed control method introduces negligible additional tracking error beyond the localization uncertainty. In addition, the curvature constraint ensures kinematically smooth motion, which is expected to reduce mechanical stress on the drivetrain and improve the predictability of robot motion. Although mechanical stress was not measured directly in this work, enforcing a curvature bound effectively increases the turning radius, which is known to reduce lateral slip and internal shear forces in skid-steer locomotion. As skid-steer-induced stress increases with tighter turning radii, curvature limitation serves as a practical mitigation mechanism.

Nevertheless, several limitations remain. The current control formulation assumes planar motion and neglects dynamic slip effects, which may become significant on deformable or inclined surfaces. Additionally, the trajectory execution layer presumes that all waypoints are reachable under the robot's curvature limits. In practice, this requires that operators avoid defining sharp reversals or excessively close points that violate geometric feasibility. Addressing such cases would require automatic trajectory regularization or online path reparameterization to enforce curvature continuity between waypoints.

Future work will extend the presented framework in several directions. First, integrating a more detailed motion model incorporating lateral slip estimation will allow adaptive adjustment of curvature limits according to terrain and surface conditions. Second, the operator interface may be expanded with semi-automated trajectory generation tools that suggest feasible paths or perform local optimization while retaining human supervision.

In conclusion, this study provides a generalized, experimentally validated framework for curvature-constrained motion control of compact heavy-duty skid-steer robots. The proposed approach achieves high accuracy and stability within the physical constraints of real-world platforms while maintaining transparency, modularity, and real-time performance. These properties make it a suitable foundation for future developments in heavy-duty mobile robotics, where precise, smooth, and geometrically feasible motion is essential for safe and reliable autonomous operation.

Author Contributions: Conceptualization, R.K. and Á.B.; methodology, software and validation, R.K.; supervision, Á.B.; project administration, Á.B. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by the European Union within the framework of the National Laboratory for Autonomous Systems (RRF-2.3.1-21-2022-00002).

Institutional Review Board Statement: Not applicable.

Data Availability Statement: All relevant data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Fragapane, G.; de Koster, R.; Sgarbossa, F.; Strandhagen, J.O. Planning and control of autonomous mobile robots for intralogistics: Literature review and research agenda. *Eur. J. Oper. Res.* **2021**, *294*, 405–426. [CrossRef]
2. Pfrommer, J.; Bömer, T.; Akizhanov, D.; Meyer, A. Sorting multibay block stacking storage systems. *arXiv* **2024**, arXiv:2405.04847. [CrossRef]
3. Abduljabbar, R.L.; Liyanage, S.; Dia, H. The role of micro-mobility in shaping sustainable cities: A systematic literature review. *Transp. Res. Part D Transp. Environ.* **2021**, *92*, 102734. [CrossRef]
4. Xidias, E.; Zacharia, P.; Nearchou, A. Intelligent fleet management of autonomous vehicles for city logistics. *Appl. Intell.* **2022**, *52*, 18030–18048. [CrossRef]
5. Paden, B.; Čáp, M.; Yong, S.Z.; Yershov, D.; Frazzoli, E. A Survey of Motion Planning and Control Techniques for Self-Driving Urban Vehicles. *IEEE Trans. Intell. Veh.* **2016**, *1*, 33–55. [CrossRef]
6. Ordonez, C.; Gupta, N.; Reese, B.; Seegmiller, N.; Kelly, A.; Collins, E.G., Jr. Learning of skid-steered kinematic and dynamic models for motion planning. *Robot. Auton. Syst.* **2017**, *95*, 207–221. [CrossRef]
7. Campion, G.; Bastin, G.; d'Andréa-Novel, B. Structural properties and classification of kinematic and dynamic models of wheeled mobile robots. *IEEE Trans. Robot. Autom.* **1996**, *12*, 47–62. [CrossRef]
8. Chen, Y.; Li, N.; Zeng, W.; Zhang, S.; Ma, G. Curved Path Following Controller for 4W Skid-Steering Mobile Robots Using Backstepping. *IEEE Access* **2022**, *10*, 66072–66082. [CrossRef]
9. Dubins, L.E. On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents. *Am. J. Math.* **1957**, *79*, 497–516. [CrossRef]
10. Qiao, Y.; He, X.; Zhuo, A.; Sun, Z.; Bao, W.; Li, Z. Curvature-Constrained Vector Field for Motion Planning of Nonholonomic Robots. *arXiv* **2025**, arXiv:2504.02852. [CrossRef]
11. Parkinson, C.; Boyle, I. Efficient and scalable path-planning algorithms for curvature-constrained motion in the Hamilton-Jacobi formulation. *J. Comput. Phys.* **2024**, *509*, 113050. [CrossRef]
12. Orthey, A.; Chamzas, C.; Kavraki, L.E. Sampling-Based Motion Planning: A Comparative Review. *Annu. Rev. Control. Robot. Auton. Syst.* **2024**, *7*, 285–310. [CrossRef]
13. Bussola, R.; Incerti, G.; Remino, C.; Tiboni, M. S-Curve Trajectory Planning for Industrial Robots Based on Curvature Radius. *Robotics* **2025**, *14*, 155. [CrossRef]
14. Chen, Z.; Shao, H.; Liu, B.; Qiao, S.; Zhou, Y.; Li, Y. FDSPC: Fast and Direct Smooth Motion Planning via Continuous Curvature Integration. *IEEE Robot. Autom. Lett.* **2025**, *10*, 10878–10885. [CrossRef]
15. Zeng, W.; Xiong, T.; Wang, C. Optimization of Smooth Trajectories for Two-Wheel Differential Robots Under Kinematic Constraints Using Clothoid Curves. *Sensors* **2025**, *25*, 3143. [CrossRef] [PubMed]
16. Foxglove. Foxglove Studio. 2024. Available online: https://foxglove.dev (accessed on 7 June 2025).
17. Krecht, R.; Ballagi, Á. Curvature-Constrained Motion Planning and Control for Traffic Cone Manipulation Robot. In Proceedings of the 22nd International Conference on Informatics in Control, Automation and Robotics (ICINCO 2025), Marbella, Spain, 20–22 October 2025; SciTePress: Setúbal, Portugal, 2025; Volume 2, pp. 317–324. [CrossRef]
18. Siciliano, B.; Sciavicco, L.; Villani, L.; Oriolo, G. *Robotics: Modelling, Planning and Control*; Springer: London, UK, 2009. [CrossRef]
19. Thrun, S.; Burgard, W.; Fox, D. *Probabilistic Robotics*; MIT Press: Cambridge, MA, USA, 2005.
20. Hanžič, F.; Jezernik, K.; Cehner, S. Mechatronic Control System on a Finite-State Machine. *Automatika* **2013**, *54*, 126–138. [CrossRef]